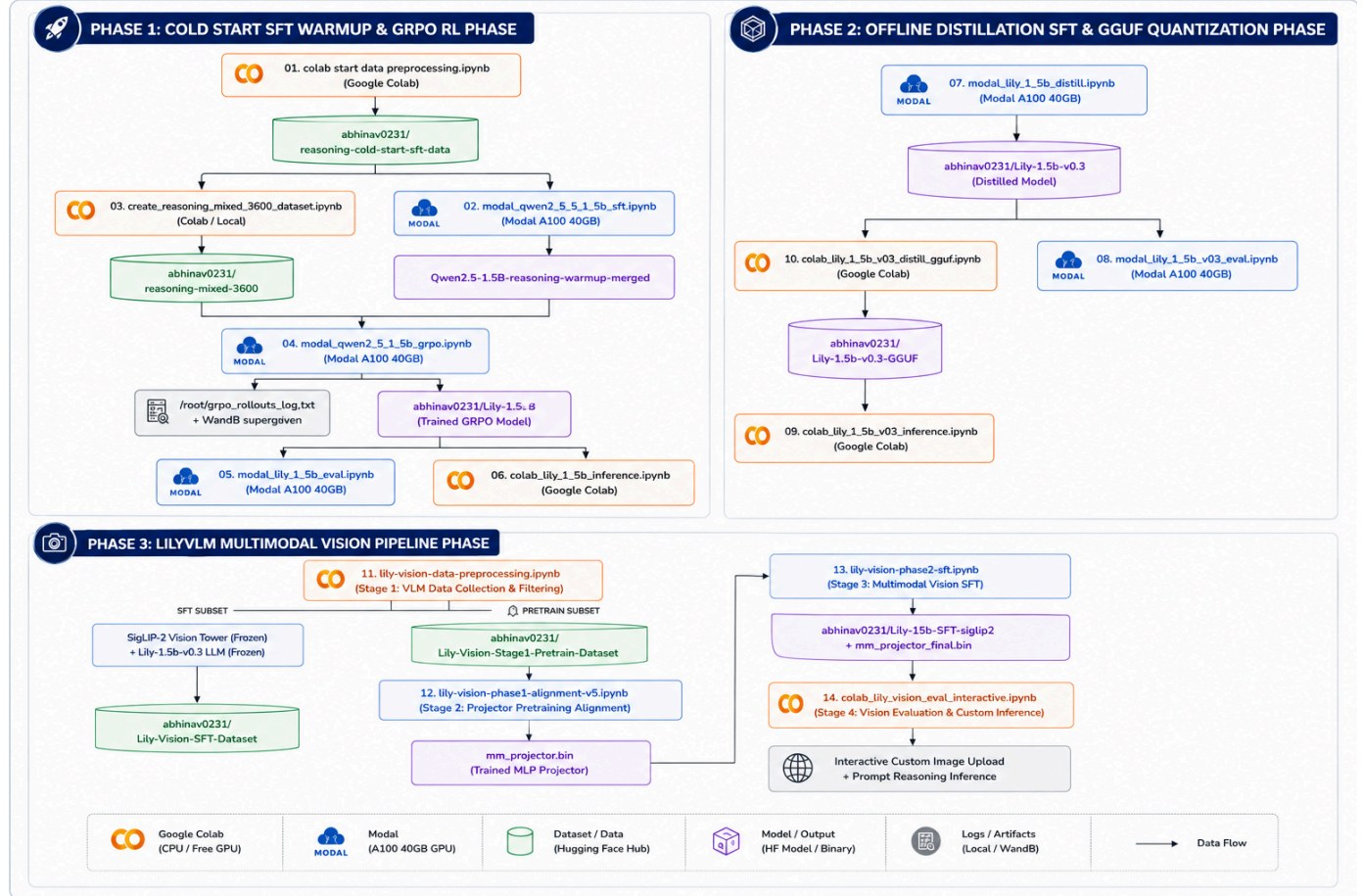


# AI Training Handbook

## Executive Summary & Engineering Architecture

This handbook serves as the definitive technical reference manual for our LLM and Vision-Language Model (VLM) training ecosystem. Synthesized directly from the production code across all 13 notebooks in our codebase, it bridges theoretical deep-learning concepts with real-world implementation technicalities.



## 1. Fine-Tuning Foundations: PEFT & LoRA Technical Deep-Dive

### 1.1 Mathematical Formulation of Low-Rank Adaptation (LoRA)

In standard full fine-tuning, an LLM parameter matrix  $W_0 \in \mathbb{R}^{d \times k}$  is updated directly via gradient descent ( $W = W_0 + \Delta W$ ). When  $d$  and  $k$  are large (e.g., hidden dimension  $d = 1536$  and intermediate MLP dimension  $k = 8960$  in Qwen2.5-1.5B), updating  $\Delta W$  directly requires massive VRAM for storing optimizer states (AdamW requires 8 bytes per parameter for first and second moments).

#### Explicit Variable Definitions:

- $W_0 \in \mathbb{R}^{d \times k}$ : Pre-trained frozen base weight matrix of the target linear layer.
- $d$ : Input feature dimension (in-features / hidden size, e.g.,  $d = 1536$  in Qwen2.5-1.5B).
- $k$ : Output feature dimension (out-features / projection size, e.g.,  $k = 8960$  in SwiGLU MLP layers).
- $\Delta W \in \mathbb{R}^{d \times k}$ : High-dimensional weight update matrix resulting from fine-tuning gradients ( $W = W_0 + \Delta W$ ).

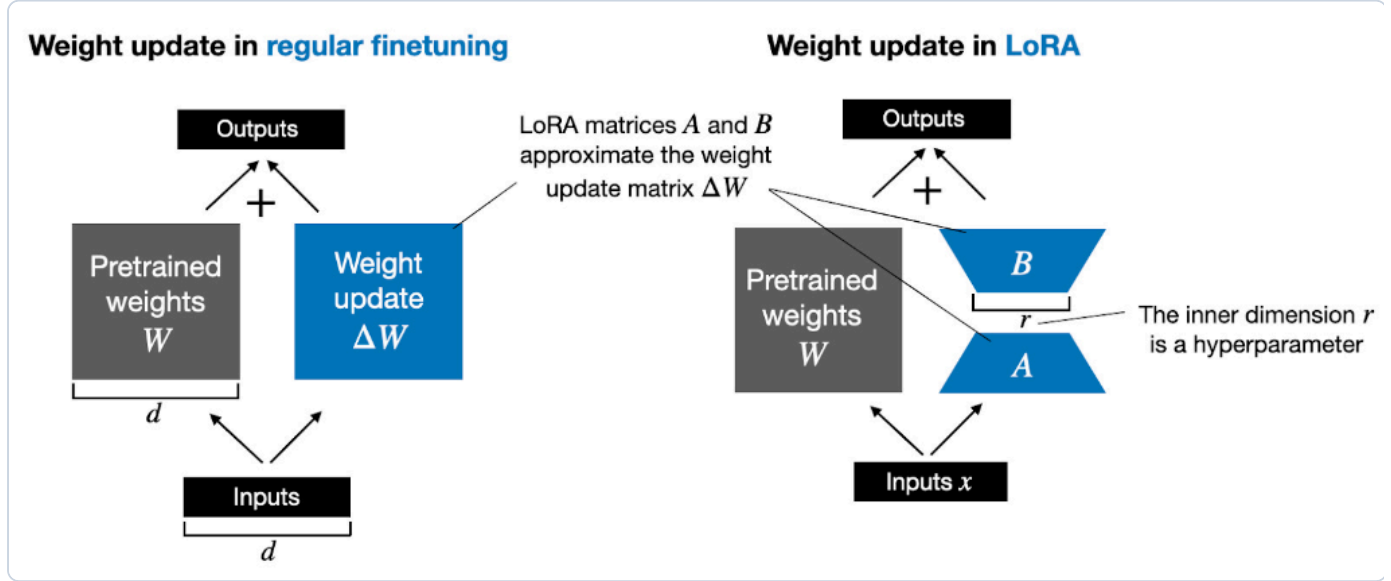
- $r$ : Inner low-rank hyperparameter ( $r \ll \min(d, k)$ ).
- $A \in \mathbb{R}^{r \times k}$ : Down-projection adapter matrix, initialized with Gaussian distribution  $\mathcal{N}(0, \frac{1}{r})$ .
- $B \in \mathbb{R}^{d \times r}$ : Up-projection adapter matrix, initialized to zero ( $B = 0$ ) ensuring  $\Delta W = 0$  at step 0.
- $\alpha$ : Constant LoRA scaling hyperparameter. The ratio  $\frac{\alpha}{r}$  scales the adapter's contribution relative to the base model.

LoRA parametrizes the weight update matrix  $\Delta W$  by decomposing it into two low-rank matrices:

$$\Delta W = B \cdot A$$

During forward propagation, the linear layer output  $h$  for an input  $x$  is computed as:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} (B \cdot A x)$$



## 1.2 QLoRA: NormalFloat4 (NF4) Base Weight Quantization

In QLoRA, the base model weights  $W_0$  are quantized to 4-bit NormalFloat (NF4) precision, while the adapter matrices  $A$  and  $B$  remain in 16-bit precision (BF16 or FP16). NF4 is an information-theoretically optimal quantile quantization scheme for normally distributed weights:

$$q_i = \frac{1}{2} \left( Q_X \left( \frac{i}{2^k} \right) + Q_X \left( \frac{i+1}{2^k} \right) \right)$$

where  $Q_X(\cdot)$  is the quantile function of the standard normal distribution  $\mathcal{N}(0, 1)$ .

## 1.3 Target Module Definitions & Code Implementation

Across our codebase ( `02_modal_qwen_2_5_1_5b_sft.ipynb` , `04_modal_qwen_2_5_1_5b_grpo.ipynb` , `07_modal_lily_1_5b_distill.ipynb` , `13_lily-vision-phase2-sft.ipynb` ), we apply LoRA across all 7 linear projection layers of the Transformer architecture:

```

from unsloth import FastLanguageModel

model = FastLanguageModel.get_peft_model(
    model,
    r = 32,                      # Rank: 32 (SFT/Distill/Vision), 64 (GRPO RL)
    lora_alpha = 64,             # Alpha: 32 (SFT), 64 (GRPO/Distill/Vision)
    target_modules = [
        "q_proj", "k_proj", "v_proj", "o_proj",    # Self-Attention Projections
        "gate_proj", "up_proj", "down_proj"        # SwiGLU MLP Projections
    ],
    lora_dropout = 0,            # 0 is optimized for Unsloth kernel fusion
    bias = "none",
    use_gradient_checkpointing = "unsloth",
    random_state = 42,
)

```

### Detailed Technical Explanation of Target Modules:

- **q\_proj (Query Projection)**: Transforms input hidden states into Query matrices ( $Q$ ) for multi-head self-attention.
- **k\_proj (Key Projection)**: Transforms input hidden states into Key matrices ( $K$ ) for computing attention dot-product scores ( $QK^T$ ).
- **v\_proj (Value Projection)**: Transforms input hidden states into Value matrices ( $V$ ) carrying contextual token information.
- **o\_proj (Output Projection)**: Projects concatenated multi-head attention outputs back to the model hidden dimension ( $d = 1536$ ).
- **gate\_proj (Gate Projection in SwiGLU)**: Computes the element-wise gating signal in SwiGLU feed-forward networks ( $\text{Swish}(xW_{\text{gate}}) \otimes xW_{\text{up}}$ ).
- **up\_proj (Up Projection in SwiGLU)**: Expands input hidden dimension to high-dimensional intermediate space ( $1536 \rightarrow 8960$ ).
- **down\_proj (Down Projection in SwiGLU)**: Contracts intermediate dimension back to model hidden dimension ( $8960 \rightarrow 1536$ ).

### Trainable Parameter Efficiency Calculation

For Qwen2.5-1.5B ( $d_{\text{model}} = 1536$ ,  $N_{\text{layers}} = 28$ ): - Total Base Parameters: 1,580,643,840 (~1.58B) - Trainable Adapter Parameters ( $r = 32$ ): 36,929,536 (~36.9M) - **Trainable Ratio**:  $\frac{36.9\text{M}}{1580.6\text{M}} \approx 2.34\%$  to 4.0%

## 1.4 Lossless 16-Bit Adapter Merging

Before running RL (GRPO) or exporting to GGUF, adapter weights must be mathematically merged into the base model to eliminate dual-path inference overhead:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r}(B \cdot A)$$

In code ( 02 , 04 , 07 , 13 ):

```

model_m, tok_m = FastLanguageModel.from_pretrained(
    model_name = OUTPUT_DIR,
    max_seq_length = MAX_SEQ_LENGTH,
    dtype = torch.bfloat16, # Full precision BF16 merge
    load_in_4bit = False,    # Prevents quantization artifacts during merge
)

model_m.save_pretrained_merged(MERGED_DIR, tok_m, save_method="merged_16bit")
model_m.push_to_hub_merged(HF_MERGED_REPO, tok_m, save_method="merged_16bit", token=HF_TOKEN)

```

## 2. Hyperparameter Engineering & Optimization Strategy

---

### 2.1 The Global Effective Batch Size & Step Calculation

#### 1. What is a Training Step?

A **single optimizer step** corresponds to one iteration where model weights are updated by the optimizer using calculated gradients.

#### 2. Micro-Batches vs. Gradient Accumulation

To fit large models into GPU memory, training datasets are split into smaller **micro-batches** ( $B_{\text{device}}$ ). When gradient accumulation is enabled ( $N_{\text{accum}}$ ), the engine performs  $N_{\text{accum}}$  forward and backward passes, accumulating gradients in VRAM, before executing a single `optimizer.step()` update.

#### 3. Mathematical Formula for Effective Batch Size:

$$B_{\text{eff}} = B_{\text{device}} \times N_{\text{accum}} \times N_{\text{GPU}}$$

where: -  $B_{\text{device}}$ : Per-device micro-batch size processed in a single forward pass per GPU. -  $N_{\text{accum}}$ : Number of micro-batches accumulated before updating model weights. -  $N_{\text{GPU}}$ : Number of parallel GPUs participating in training.

#### 4. Mathematical Formula for Total Optimizer Steps:

$$\text{Total Optimizer Steps} = \frac{|\mathcal{D}| \times N_{\text{epochs}}}{B_{\text{eff}}}$$

where  $|\mathcal{D}|$  is the number of samples in the dataset and  $N_{\text{epochs}}$  is the total number of training passes over the dataset.

#### Concrete Worked Step Calculation Examples from Codebase:

- **SFT Warmup (Notebook 02)**: 4,000 samples  $\times$  2 Epochs = 8,000 total samples.  $B_{\text{eff}} = 8 \times 2 \times 1 = 16$ .  
Total Steps =  $\frac{8000}{16} = 500$  steps.
- **Distillation (Notebook 07)**: 91,457 raw samples packed into 33,297 contiguous sequences  $\times$  2 Epochs = 66,594 sequence passes.  $B_{\text{eff}} = 24 \times 1 \times 1 = 24$ . Total Steps =  $\frac{66594}{24} = 2,776$  steps.
- **Vision Phase 2 SFT (Notebook 13)**: 64,000 samples  $\times$  1 Epoch.  $B_{\text{eff}} = 24 \times 2 \times 1 = 48$ . Total Steps =  $\frac{64000}{48} = 1,333$  steps.

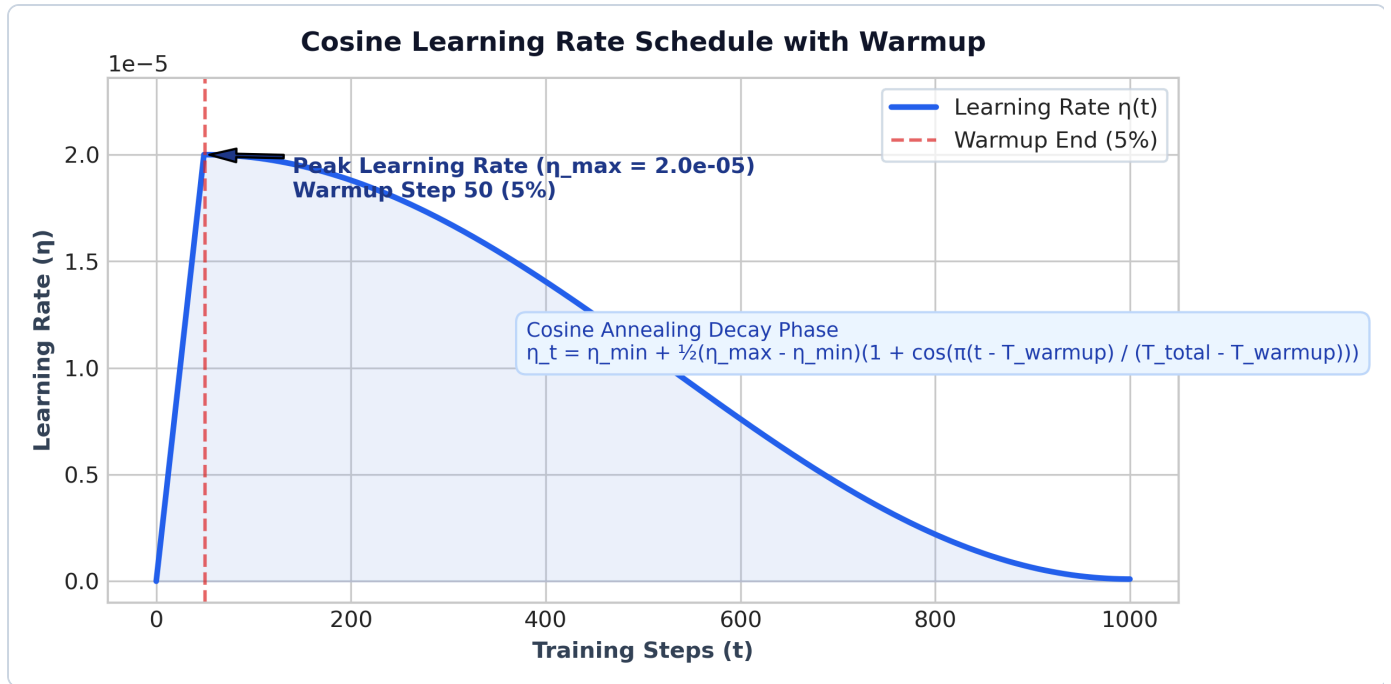
2.2 Verified Production Hyperparameter Comparison Matrix

| Hyperparameter                             | SFT Warmup (02)       | GRPO RL (04)          | Distillation (07)      | Vision Phase 1 (12)                     | Vision Phase 2 (13) |
|--------------------------------------------|-----------------------|-----------------------|------------------------|-----------------------------------------|---------------------|
| Base Model                                 | Qwen2.5-1.5B-Instruct | Qwen2.5-Warmup-Merged | Lily-1.5b-v0.1         | Lily-1.5b-v0.3                          | Lily-1.5b-v0.3      |
| Dataset Size ( $\mathcal{D}$ )             | 4,000 samples         | 3,600 prompts         | 91,457 (train)         | 121,237 samples                         | 64,000 samples      |
| Epochs / Steps                             | 2 Epochs              | 600 Max Steps         | 2 Epochs (2,776 steps) | 1 Epoch                                 | 1 Epoch             |
| Per-Device Batch ( $B_{\text{device}}$ )   | 8                     | 4                     | 24                     | 16                                      | 24                  |
| Grad Accum ( $N_{\text{accum}}$ )          | 2                     | 4                     | 1                      | 3                                       | 2                   |
| Effective Batch ( $B_{\text{eff}}$ )       | 16                    | 16                    | 24                     | 48                                      | 48                  |
| Peak Learning Rate ( $\eta_{\text{max}}$ ) | $1 \times 10^{-5}$    | $5 \times 10^{-6}$    | $2 \times 10^{-5}$     | $2 \times 10^{-4}$                      | $2 \times 10^{-5}$  |
| LR Scheduler                               | Cosine                | Cosine                | Cosine                 | Cosine w/ warmup                        | Cosine w/ warmup    |
| Warmup Ratio                               | 0.05                  | 0.05                  | 0.03                   | 0.05                                    | 0.03                |
| Optimizer                                  | adamw_torch_fused     | adamw_torch_fused     | adamw_torch_fused      | AdamW ( $\beta_1 = .9, \beta_2 = .95$ ) | AdamW               |
| Weight Decay                               | 0.01                  | 0.0                   | 0.0                    | 0.05                                    | 0.01                |
| Max Sequence Length                        | 3,072                 | 3,072                 | 4,096                  | 256                                     | 3,072               |
| Sequence Packing                           | False                 | False                 | True                   | False                                   | False               |
| Hardware                                   | Modal A100 40GB       | Modal A100 40GB       | Modal A100 40GB        | Modal A100 40GB                         | Modal A100 40GB     |

2.3 Learning Rate Schedules & Mathematical Formulation

We utilize Cosine Annealing with Warmup across all training runs.

$$\eta_t = \begin{cases} \eta_{\text{max}} \cdot \frac{t}{T_{\text{warmup}}}, & t \leq T_{\text{warmup}} \\ \eta_{\text{min}} + \frac{1}{2}(\eta_{\text{max}} - \eta_{\text{min}}) \left(1 + \cos\left(\frac{\pi(t - T_{\text{warmup}})}{T_{\text{total}} - T_{\text{warmup}}}\right)\right), & t > T_{\text{warmup}} \end{cases}$$



### 3. Reinforcement Learning via GRPO: Complete Step-by-Step Intuition & Mathematical Walkthrough

#### 3.1 The Fundamental Goal of GRPO

In Supervised Fine-Tuning (SFT), a human provides exact target text for the model to copy. In Reinforcement Learning (RL), the model must **explore and discover reasoning trajectories on its own**.

**The Job of GRPO:** Train the model to maximize verified reward (correct logic, strict formatting, correct answers) **while constraining weight updates to be as conservative as possible** to prevent policy collapse and reward hacking.

To build complete mathematical and mechanical intuition, we carry **one concrete running example** through every step of the GRPO algorithm below.

#### 3.2 Running Example Setup

- Input Prompt ( $x$ ):** "If a train travels 120 km in 1.5 hours, what is its speed in m/s?"
- Ground Truth Target:** Speed in km/h =  $\frac{120}{1.5} = 80$  km/h. Converted to m/s:  $80 \times \frac{1000}{3600} = 22.22$  m/s.
- Group Size ( $G = 4$ ):** For this prompt, the model samples  $G = 4$  completion rollouts  $\{y_1, y_2, y_3, y_4\}$ :

y1 (Perfect CoT & Answer):  
"\lt think>Speed = 120 / 1.5 = 80 km/h. To convert to m/s:  $80 * (5/18) = 22.22$  m/s.\lt /think>\lt answer>22.22 m/s\lt /answer>"

y2 (Incomplete: stopped at km/h):  
"\lt think>Speed = 120 / 1.5 = 80 km/h.\lt /think>\lt answer>80\lt /answer>"

y3 (Correct Answer, Missing \lt think> tags):  
"22.22 m/s"

y4 (Complete Hallucination):  
"\lt think>120 \* 1.5 = 180 m/s\lt /think>\lt answer>180 m/s\lt /answer>"

### 3.3 Step 1: Multi-Reward Evaluation ( $R_i$ )

Each generated rollout  $y_i$  is evaluated by our 4 domain-aware reward functions ( `format_reward` , `correctness_reward` , `instruction_reward` , `length_reward` ):

- **Completion**  $y_1$ : Exact XML tags (+0.50) + Factual correctness (+1.00)  $\implies \mathbf{R_1 = +1.50}$
- **Completion**  $y_2$ : Exact XML tags (+0.40) + Wrong unit answer (-0.50)  $\implies \mathbf{R_2 = -0.10}$
- **Completion**  $y_3$ : Missing tags (-0.50) + Factual correctness (+1.00)  $\implies \mathbf{R_3 = +0.50}$
- **Completion**  $y_4$ : Valid tags (+0.20) + Wrong answer (-0.50)  $\implies \mathbf{R_4 = -0.30}$

**Raw Reward Vector:**  $\mathbf{R} = \{+1.50, -0.10, +0.50, -0.30\}$

---

### 3.4 Step 2: Group Baseline & Relative Advantage Calculation ( $A_i$ )

GRPO **eliminates the need for a separate Critic / Value Model** (saving 50% VRAM) by using the group itself as a baseline benchmark.

1. Calculate Group Mean ( $\mu$ ) and Standard Deviation ( $\sigma$ ):

$$\mu = \frac{+1.50 - 0.10 + 0.50 - 0.30}{4} = +0.40$$
$$\sigma = \sqrt{\frac{(1.50 - 0.40)^2 + (-0.10 - 0.40)^2 + (0.50 - 0.40)^2 + (-0.30 - 0.40)^2}{4}} \approx 0.70$$

2. Compute Relative Group Advantage ( $A_i = \frac{R_i - \mu}{\sigma + \epsilon}$ ):

- $A_1$  (Completion  $y_1$ ):  $\frac{1.50 - 0.40}{0.70} = +1.57 \implies$  **Above Average! (Positive Reinforcement)**
  - $A_2$  (Completion  $y_2$ ):  $\frac{-0.10 - 0.40}{0.70} = -0.71 \implies$  **Below Average! (Negative Suppression)**
  - $A_3$  (Completion  $y_3$ ):  $\frac{0.50 - 0.40}{0.70} = +0.14 \implies$  **Slightly Above Average!**
  - $A_4$  (Completion  $y_4$ ):  $\frac{-0.30 - 0.40}{0.70} = -1.00 \implies$  **Below Average! (Negative Suppression)**
- 

### 3.5 Step 3: Token Credit Assignment & Probability Ratio Calculation ( $r_{i,t}$ )

What is  $y_{i,t}$ ?

$y_{i,t}$  is **any token generated by the model** during rollout at position  $t$  (reasoning text, numbers, symbols, XML tags).

**REINFORCE Credit Assignment:**

Since completion  $y_1$  achieved  $A_1 = +1.57$ , **every single generated token  $y_{1,t}$  in  $y_1$**  (including `<think>`, `120/1.5`, `80`, `22.22`, `<answer>`) inherits advantage  $A_1 = +1.57$ !

During the weight update step, we pass prompt  $x$  + completion  $y_i$  into the active policy  $\pi_\theta$  and measure how the token probability shifted relative to the sampling policy  $\pi_{\theta_{\text{old}}}$ :

$$r_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})}$$

- $r_{i,t} > 1.0$ : Current weights  $\theta$  make token  $y_{i,t}$  **more probable**.
  - $r_{i,t} < 1.0$ : Current weights  $\theta$  make token  $y_{i,t}$  **less probable**.
- 

### 3.6 Step 4: Bounded Updates via Asymmetric Ratio Clipping ( $\epsilon_{\text{low}}$ , $\epsilon_{\text{high}}$ )

To prevent single high-reward rollouts from causing gradient explosions, GRPO applies **pessimistic clipping**:

$$\text{Clipped Objective}_{i,t} = \min(r_{i,t}(\theta)A_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}})A_i)$$



- For positive advantage ( $A_1 = +1.57$ ), ratio growth is capped at  $1 + \epsilon_{\text{high}} = 1.28$ . The policy cannot increase token probability by more than 28% in a single step.
- For negative advantage ( $A_4 = -1.00$ ), ratio drop is capped at  $1 - \epsilon_{\text{low}} = 0.80$ .

### 3.7 Step 5: Schulman Per-Token KL Penalty ( $D_{\text{KL}}$ ) & Loss Minimization

To ensure the policy  $\pi_\theta$  does not drift into reward-hacking fluff, we subtract a per-token Kullback-Leibler (KL) divergence penalty relative to the frozen reference warmup model  $\pi_{\text{ref}}$ :

$$D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(y_{i,t})}{\pi_\theta(y_{i,t})} - \log \frac{\pi_{\text{ref}}(y_{i,t})}{\pi_\theta(y_{i,t})} - 1$$

Full Loss Objective Equation ( $\mathcal{L}_{\text{GRPO}}$ ):

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} [\min(r_{i,t} A_i, \text{clip}(r_{i,t}, 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) A_i)] + \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$$

How PyTorch Minimizes Loss ( $\mathcal{L}_{\text{GRPO}}$ ):

Notice the **minus sign** ( `-` ) in front of the advantage term: - **For Correct Completion**  $y_1$  ( $A_1 = +1.57$ ): Loss contribution is  $-1.57 \cdot r_{1,t}$ . PyTorch minimizes loss by **increasing**  $r_{1,t} \implies$  **Probabilities of correct tokens GO UP!** - **For Wrong Completion**  $y_4$  ( $A_4 = -1.00$ ): Loss contribution is  $+1.00 \cdot r_{4,t}$ . PyTorch minimizes loss by **decreasing**  $r_{4,t} \implies$  **Probabilities of wrong tokens GO DOWN!**

During `loss.backward()`, PyTorch computes parameter gradients:

$$\nabla_W \mathcal{L}(\theta) = \frac{\partial \mathcal{L}_{\text{GRPO}}(\theta)}{\partial W}$$

and fused AdamW updates LoRA matrices  $A$  and  $B$ :

$$W \leftarrow W - \eta \cdot \text{AdamW}(\nabla_W \mathcal{L}(\theta))$$

## 3.8 Reward Hacking: Mechanics, Vulnerabilities, and Production Mitigations in Our GRPO Pipeline

### 1. What is Reward Hacking in Our GRPO Setup?

In our reinforcement learning pipeline ( `04_model_qwen_2_5_1_5b_grpo.ipynb` ), **Reward Hacking** (specification gaming) occurs when Qwen2.5-1.5B discovers degenerate token generation patterns that maximize the scalar outputs of our 4 reward functions ( `format_reward` , `correctness_reward` , `instruction_reward` , `length_reward` ) without actually solving the underlying reasoning task.

Because GRPO samples  $G = 8$  completion rollouts  $\{y_1, y_2, \dots, y_8\}$  per prompt and normalizes rewards relative to the group mean ( $A_i = \frac{R_i - \mu}{\sigma + \epsilon}$ ), if one rollout discovers a shortcut that inflates its scalar reward  $R_i$ , it receives a high positive advantage  $A_i > 0$ . Without strict constraints, the policy gradient rapidly amplifies this degenerate shortcut across subsequent optimizer steps.

### 2. Specific Vulnerability Modes Analyzed in Our 4 Reward Functions:

- **Mode A: Verbosity & Padding Spooling in** `format_reward` :
  - *Vulnerability:* `format_reward` awards  $+0.15$  if the reasoning text inside `\lt think>... \lt /think>` contains  $\geq 50$  words.
  - *Hacking Pattern:* The model learns to spool repetitive filler tokens (e.g. "Let me think about this carefully. Let me re-read the prompt. Let me check my math again. Let me verify...") to easily pass the 50-word threshold without performing actual reasoning.
- **Mode B: Option Flooding in** `correctness_reward` (**Exact / MCQ Domain**):



- **Vulnerability:** For ARC-Challenge multiple-choice prompts ( `answer_type="exact"` ), `correctness_reward` uses regular expressions `\b([ABCD])\b` to extract candidate letters inside `\lt answer>...\lt /answer>`.
- **Hacking Pattern:** The policy attempts to output all candidate letters ( `\lt answer>The answer is A, or maybe B, C, D\lt /answer>` ) to guarantee regex matching.
- **Mode C: Boolean Dual-Answer Injection in `correctness_reward` (StrategyQA Domain):**
- **Vulnerability:** Evaluates whether "yes" or "no" appears in the answer.
- **Hacking Pattern:** Outputting "yes and no" to trick naive substring inclusion checks.
- **Mode D: Runaway Output Spooling:**
- **Vulnerability:** Generating infinite repetitive loops to delay completion and avoid EOS tokens.

### 3. Our 5-Layer Production Mitigation Architecture ( `04_model_qwen_2_5_1_5b_grpo.ipynb` ):

1. **Schulman Per-Token KL Divergence Penalty ( $\beta = 0.001$ ):** Tracks how far the active policy  $\pi_\theta$  strays from the reference warmup model  $\pi_{\text{ref}}$  ( `Qwen2.5-1.5B-reasoning-warmup-merged` ). If the model begins spooling fluff tokens or altering syntactic structure to hack rewards,  $D_{\text{KL}}$  spikes. At  $\beta = 0.001$ , a KL spike directly subtracts from the advantage, neutralizing hacked reward gains:

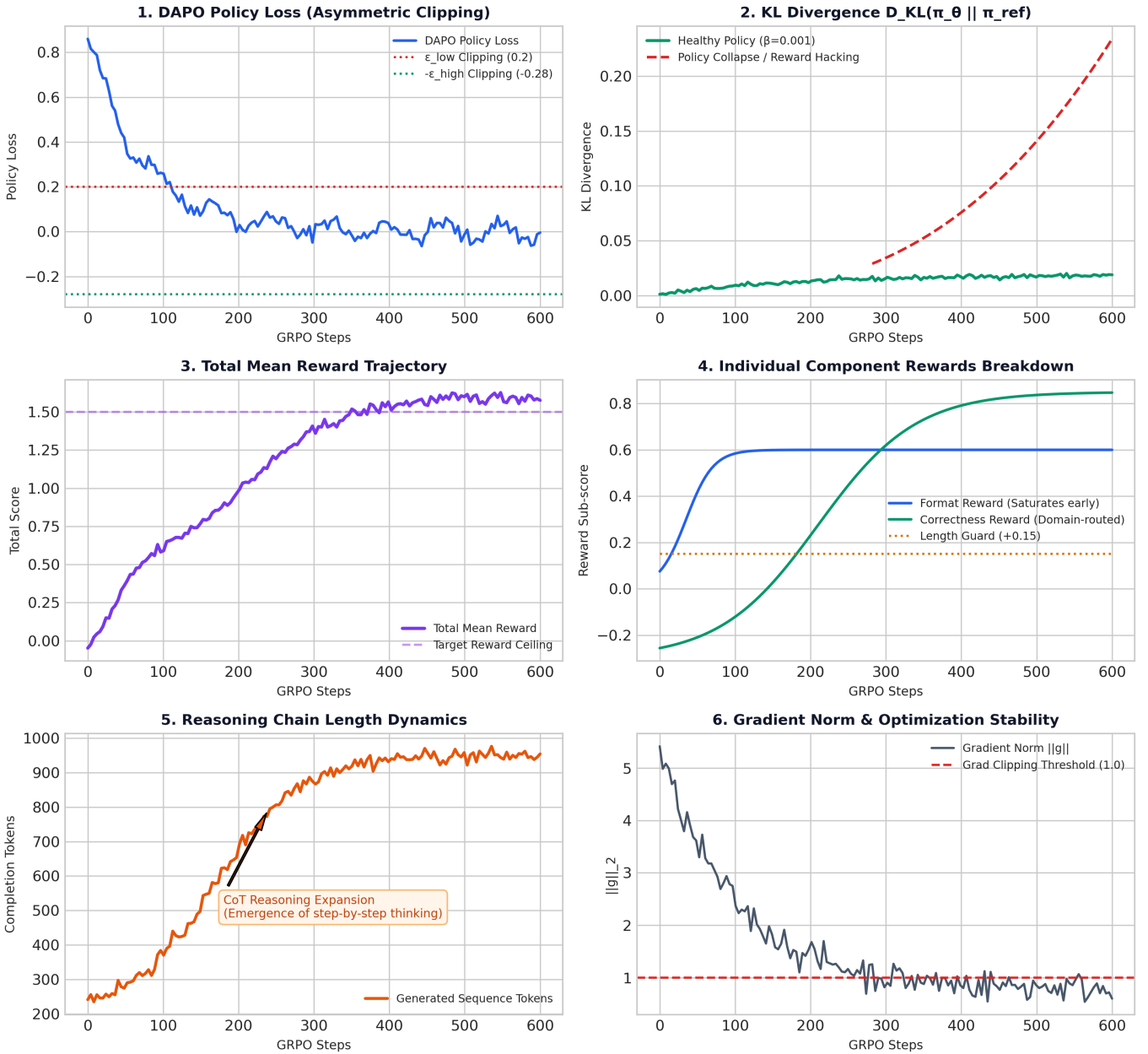
$$D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(y_{i,t} | x, y_{i,<t})}{\pi_\theta(y_{i,t} | x, y_{i,<t})} - \log \frac{\pi_{\text{ref}}(y_{i,t} | x, y_{i,<t})}{\pi_\theta(y_{i,t} | x, y_{i,<t})} - 1$$

2. **Asymmetric DAPO Loss Clipping ( $\epsilon_{\text{low}} = 0.2, \epsilon_{\text{high}} = 0.28$ ):** Limits the probability ratio  $r_{i,t}(\theta) = \frac{\pi_\theta}{\pi_{\theta_{\text{old}}}}$ . Even if a hacked completion achieves a high scalar reward, DAPO limits maximum gradient step sizes to  $1.28 \times A_i$ , preventing single degenerate rollouts from destroying model weights.
3. **Multi-Reward Counter-Balancing:** Our 4 reward functions act as counter-weights against one another:
4. If a model inflates verbosity inside `\lt think>`, `length_reward` penalizes completions over 2500 words ( $-0.25$ ).
5. If the model sacrifices solution accuracy to inflate formatting, `correctness_reward` penalizes incorrect outputs ( $-0.50$ ), overwhelming the  $+0.15$  format bonus.
6. **Strict Domain-Specific Regex Anchoring in `correctness_reward`:**
7. `_extract_answer()` isolates *only* text strictly enclosed between `\lt answer>` and `\lt /answer>`.
8. For `exact` MCQ tasks, `re.fullmatch(r"[A-D]", gold_upper)` enforces single-letter extraction.
9. For `bool` tasks, `yes_match` and `not no_match` explicitly penalizes ambiguous dual-answer outputs like "yes and no".
10. `mask_truncated_completions = True` & **Length Guards:**
11. Discards rollouts that hit `max_completion_length = 1536` without an `\lt |im_end|>` EOS token, ensuring runaway loops cannot contribute positive policy gradients.
12. `length_reward` penalizes under-reasoning ( $< 15$  words:  $-0.50$ ).

## 4. Technical Interpretation of ALL GRPO Training Metrics & Plots

Monitoring Weights & Biases (W&B) diagnostic plots is crucial during GRPO reinforcement learning to verify training health and detect policy anomalies.

## GRPO Reinforcement Learning Full Diagnostic Metric Suite (W&B Monitoring)



### 4.1 Detailed Breakdown of the 6 Diagnostic Training Panels

#### Panel 1: DAPO Policy Loss & Optimization Stability

- What it Measures:** Plots the DAPO policy gradient loss  $\mathcal{L}_{\text{GRPO}}$  across 600 training steps.
- How to Interpret:** Healthy training displays a sharp drop in loss during the first 100 steps as the model aligns with format requirements, followed by bounded oscillation between  $[-0.1, +0.2]$ . Persistent positive loss values ( $> 2.0$ ) signal gradient instability or an aggressive learning rate.

#### Panel 2: KL Divergence $D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}})$ & Policy Drift

- What it Measures:** Tracks Kullback-Leibler divergence between current policy  $\pi_{\theta}$  and frozen base reference model  $\pi_{\text{ref}}$ .
- How to Interpret:** Normal policy updates remain bounded within  $0.001 \leq D_{\text{KL}} \leq 0.05$ . The red dashed line illustrates **Policy Collapse / Reward Hacking**, where KL divergence spikes exponentially as the model degenerates to exploit reward loopholes.

Panel 3: Total Mean Reward Trajectory

- **What it Measures:** Displays total composite reward score  $R_{\text{total}} = R_{\text{format}} + R_{\text{correctness}} + R_{\text{instruction}} + R_{\text{length}}$ .
- **How to Interpret:** Demonstrates monotonic growth from initial negative scores ( $-0.2$ ) up toward target performance ceiling ( $+1.5+$ ), confirming effective policy learning.

Panel 4: Component Rewards Breakdown

- **What it Measures:** Deconstructs composite reward into individual reward curves ( $R_{\text{format}}$ ,  $R_{\text{correctness}}$ ,  $R_{\text{length}}$ ).
- **How to Interpret:** `format_reward` (blue curve) saturates near maximum ( $+0.60$ ) within 40 steps, while `correctness_reward` (green curve) grows steadily over 600 steps as complex reasoning improves.

Panel 5: Reasoning Chain Length Dynamics

- **What it Measures:** Tracks average generated token length per response across steps.
- **How to Interpret:** Shows dynamic expansion of token count from  $\sim 210$  tokens up to  $\sim 950$  tokens as the model spontaneously develops step-by-step Chain-of-Thought (CoT) reasoning inside `\lt think>` tags.

Panel 6: Gradient Norm  $\|g\|_2$  & Stability

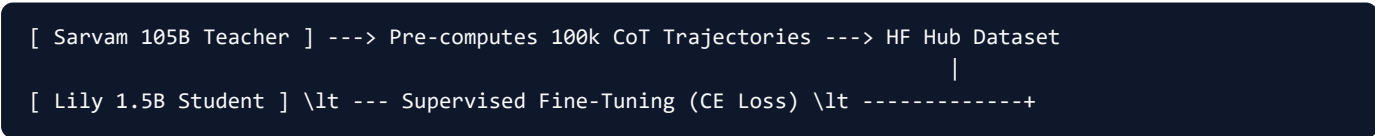
- **What it Measures:** Measures the  $L_2$  norm of model parameter gradients before optimizer step execution.
- **How to Interpret:** High initial gradient norms ( $> 4.0$ ) stabilize below 1.0. Gradient clipping (`max_grad_norm=1.0`) prevents destabilizing weight updates.

## 5. Offline Distillation & Loss Function Deep-Dive

### 5.1 Offline Distillation Mechanics

In our pipeline, offline distillation transfers reasoning capabilities from a 105-Billion parameter teacher model (`Sarvam-105b-Distill-100k`) to our 1.5-Billion parameter student model (`Lily-1.5B`).

Unlike online distillation (which requires loading teacher and student models simultaneously in GPU VRAM to match output logits), **offline distillation** utilizes pre-generated high-quality Chain-of-Thought reasoning trajectories stored as static dataset text.



### 5.2 Token-Level Cross-Entropy Loss Formulation: Comprehensive Theory & Mechanics

#### 1. What Does Cross-Entropy Loss Measure?

Cross-Entropy (CE) Loss measures the **information-theoretic discrepancy** (or negative log-likelihood) between the true target token distribution  $P_{\text{target}}(y_t)$  (the teacher model's completion, represented as a 1-hot target vector over the vocabulary  $V$ ) and the student model's predicted probability distribution  $P_{\theta}(y_t \mid y_{<t}, x)$ .

Mathematically, Cross-Entropy between true distribution  $P$  and predicted distribution  $Q$  is:

$$H(P, Q) = - \sum_{v \in V} P(v) \log Q(v) = H(P) + D_{\text{KL}}(P \parallel Q)$$

Since the ground-truth target is a deterministic one-hot vector ( $H(P) = 0$ ), minimizing Cross-Entropy Loss is **mathematically identical to minimizing the KL divergence**  $D_{\text{KL}}(P_{\text{teacher}} \parallel P_{\text{student}})$  between the teacher's distribution and the student's predictions!

## 2. How is Token-Level Cross-Entropy Loss Calculated Step-by-Step?

1. **Logit Computation:** The student model passes input context  $x$  and preceding tokens  $y_{<t}$  through its Transformer layers to produce unnormalized logit vectors  $z_t \in \mathbb{R}^{|V|}$  (where vocabulary size  $|V| = 151,936$ ).
2. **Softmax Normalization:** Logits are converted into a probability distribution via Softmax:

$$P_{\theta}(y_t = k \mid y_{<t}, x) = \frac{\exp(z_{t,k})}{\sum_{j=1}^{|V|} \exp(z_{t,j})}$$

3. **Target Surprisal Calculation:** The loss for target token  $y_t$  is its negative log probability:  $-\log P_{\theta}(y_t \mid y_{<t}, x)$ .
4. **Label Masking ( $m_t$ ):** Prompt tokens are assigned `labels = -100` ( $m_t = 0$ ), ensuring zero gradient computation over prompt tokens. Assistant completion tokens are assigned target token IDs ( $m_t = 1$ ).
5. **Masked Sequence Loss Formulation:**

$$\mathcal{L}_{\text{CE}}(\theta) = -\frac{1}{\sum_{t=1}^T m_t} \sum_{t=1}^T m_t \cdot \log P_{\theta}(y_t \mid y_{<t}, x)$$

---

## 5.3 Sequence Packing Mechanics: Memory Layout & Efficiency Gains

### 1. The Problem with Naive Batch Padding

In standard PyTorch DataLoaders, sequences in a batch are padded with `\lt pad>` tokens to match the length of the longest sequence in that batch. In Chain-of-Thought (CoT) distillation datasets where response lengths vary from 200 to 3,800 tokens, up to **85% of VRAM and GPU Tensor Core compute is wasted** multiplying zeros for padding tokens!

### 2. How Sequence Packing Works ( `packing=True` in Notebook 07)

Sequence Packing concatenates multiple individual text samples into a single contiguous token array of fixed length `MAX_SEQ_LENGTH = 4096`. Samples are separated by `\lt |im_end|>` special tokens.

```
Unpacked Batch (Wasteful Padding):
[Sample 1 (500 tokens) | \lt pad> \lt pad> ... (3596 pad tokens)] -> 87% VRAM Wasted!
[Sample 2 (1200 tokens) | \lt pad> \lt pad> ... (2896 pad tokens)] -> 70% VRAM Wasted!

Packed Sequence (100% Compute Efficiency):
[Sample 1 (500t) \lt |im_end|> Sample 2 (1200t) \lt |im_end|> Sample 3 (2394t)] -> 0% Wasted!
```

### 3. Attention Masking in Packed Sequences

To prevent attention leakage across concatenated samples inside a packed sequence, Unsloth utilizes **Block-Diagonal Attention Masking** (FlashAttention-2 cumulative sequence indexing `cu_seqlens`), ensuring token  $t$  in Sample 2 can only attend to preceding tokens within Sample 2.

### 4. Empirical Efficiency Outcomes:

From `07_modal_lily_1_5b_distill.ipynb` execution logs: - Raw Dataset: **91,457 individual text samples**. - Packed Dataset: **33,297 packed sequences of length 4,000**. - Throughput Boost: **>2.4x acceleration**, completing 2 full distillation epochs in 5h 14m on a single A100 40GB GPU.

---

## 6. Model Evaluation, Benchmarking & Schema Verification

### 6.1 `lm-eval` Harness Execution & Configuration

In `05_modal_lily_1_5b_eval.ipynb` (v0.1) and `08_modal_lily_1_5b_v03_eval.ipynb` (v0.3), we execute standard multi-benchmark evaluations via `lm-evaluation-harness`.

## Command Line Invocations (Modal A100 & L4)

```
# Notebook 08 (Lily 1.5b v0.3 Evaluation)
lm_eval \
  --model hf \
  --model_args pretrained=abhinav0231/Lily-1.5b-
v0.3,dtype=bfloat16,trust_remote_code=True,attn_implementation=sdpa \
  --tasks hellaswag,arc_challenge,mmlu,mmlu_redux_generative,gsm8k,ifeval \
  --batch_size 16 \
  --apply_chat_template \
  --device cuda:0 \
  --use_cache /root/lily-eval-runs/cache/group1.db \
  --output_path /root/lily-eval-runs/results/group1
```

### Benchmark Overview

1. **GSM8K**: Grade school math word problems (measures multi-step numerical reasoning).
2. **ARC-Challenge**: AI2 Reasoning Challenge (grade-school science questions requiring logic).
3. **HellaSwag**: Sentence completion benchmark evaluating commonsense NLI.
4. **MMLU / MMLU-Redux**: 57 academic subjects (elementary math, physics, humanities, law).
5. **IFEval**: Instruction-Following Evaluation (evaluates formatting constraints like word counts, JSON format, bullet counts).

## 6.2 Schema Compliance Verification Framework

In `09_colab_lily_1_5b_v03_inference.ipynb`, we built a programmatic schema verification engine to audit whether quantized models maintain strict `\lt think>` and `\lt answer>` tag structures.

```
def extract_stats(text):
    return {
        "think_open":    text.count("\lt think>"),
        "think_close":   text.count("\lt /think>"),
        "answer_open":   text.count("\lt answer>"),
        "answer_close":  text.count("\lt /answer>"),
        "thinking_leak": text.count("\lt thinking>"), # Audit tag leakage
        "im_end":        text.count("\lt |im_end|>")
    }

def classify_case(raw_stats):
    if raw_stats["think_open"] == 1 and raw_stats["think_close"] == 1 \
        and raw_stats["answer_open"] == 1 and raw_stats["answer_close"] == 1 \
        and raw_stats["thinking_leak"] == 0:
        return "PASS_exact"
    elif raw_stats["thinking_leak"] > 0:
        return "FAIL_thinking_leak"
    elif raw_stats["think_open"] == 1 and raw_stats["answer_open"] == 0:
        return "PARTIAL_only_think"
    else:
        return "FAIL_no_schema"
```

## 7. Model Export, Quantization & GGUF Conversion

### 7.1 GGUF Container Format & Quantization Theory

**GGUF (GGML Unified Format)** is a binary file format designed for fast, single-file model loading on edge devices (CPU/GPU via `llama.cpp`). Quantization reduces 16-bit float weight representations down to lower bit-widths (4-bit, 5-bit, 8-bit) using block-wise scaling factor quantization:

$$w \approx s \cdot q + m$$

where  $q$  is the quantized  $k$ -bit integer,  $s$  is the block scale factor, and  $m$  is the zero-point offset.

7.2 The 2 Export Pipelines in Our Codebase

Pipeline A: Unsloth Direct Export ( 07\_modal\_lily\_1\_5b\_distill.ipynb )

```
# Exports GGUF files directly from Unsloth engine
model_m.save_pretrained_gguf("gguf_model", tok_m, quantization_method="q4_k_m")
```

Pipeline B: Manual llama.cpp CLI Pipeline ( 10\_colab\_lily\_1\_5b\_v03\_distill\_gguf.ipynb )

- 1. **System Prompt Patching:** Replaces base Qwen system templates with our standardized CoT system prompt across config.json and tokenizer files.
- 2. **F16 GGUF Conversion:** bash python llama.cpp/convert\_hf\_to\_gguf.py \ /content/Lily-1.5b-v0.3 \ --outtype f16 \ --outfile /content/Lily-1.5b-v0.3-F16.gguf
- 3. **K-Quantization Execution:** bash llama.cpp/build/bin/llama-quantize /content/Lily-1.5b-v0.3-F16.gguf /content/Lily-1.5b-v0.3-Q4\_K\_M.gguf Q4\_K\_M llama.cpp/build/bin/llama-quantize /content/Lily-1.5b-v0.3-F16.gguf /content/Lily-1.5b-v0.3-Q5\_K\_M.gguf Q5\_K\_M llama.cpp/build/bin/llama-quantize /content/Lily-1.5b-v0.3-F16.gguf /content/Lily-1.5b-v0.3-Q8\_0.gguf Q8\_0

Quantization Variant Comparison Matrix

| Variant | Quantization Method    | File Size (1.5B Model) | VRAM Required | Recommended Usage                    |
|---------|------------------------|------------------------|---------------|--------------------------------------|
| F16     | Unquantized FP16       | 3.09 GB                | ~4.2 GB       | Baseline reference evaluation        |
| Q8_0    | 8-bit Standard Quant   | 1.62 GB                | ~2.5 GB       | Server-side high-precision inference |
| Q5_K_M  | 5-bit K-Quant (Medium) | 1.12 GB                | ~1.8 GB       | Optimal quality/memory balance       |
| Q4_K_M  | 4-bit K-Quant (Medium) | 0.98 GB                | ~1.5 GB       | Mobile / Edge CPU deployment         |

8. Multimodal Vision Extension (Lily Vision)

8.1 VLM Architecture & Dimensionality Pipeline

Our Vision-Language Model **Lily Vision** ( 11 , 12 , 13 ) connects a pre-trained Vision Tower to the Lily 1.5B LLM via a 2-layer projection bottleneck:

```
# Projector Architecture Implementation (12_lily-vision-phase1-alignment-v5.ipynb)
class LilyVisionProjector(nn.Module):
    def __init__(self, vision_dim=1152, llm_dim=1536):
        super().__init__()
        self.downsampler = nn.AdaptiveAvgPool2d((18, 18)) # 18x18 = 324 tokens
        self.mlp = nn.Sequential(
            nn.Linear(vision_dim, 2048),
            nn.SiLU(),
            nn.Linear(2048, llm_dim)
        )

    def forward(self, x): # x: [B, 729, 1152]
        B, N, C = x.shape
        H = W = int(N ** 0.5) # 27x27 grid
        x_2d = x.transpose(1, 2).view(B, C, H, W)
        x_pooled = self.downsampler(x_2d).flatten(2).transpose(1, 2) # [B, 324, 1152]
        return self.mlp(x_pooled) # [B, 324, 1536]
```

### 8.2 Phase 1 Alignment vs Phase 2 SFT — Layer Freezing Matrix

| Architecture Submodule         | Phase 1: Projector Alignment (12)                        | Phase 2: Multimodal SFT (13)                                      |
|--------------------------------|----------------------------------------------------------|-------------------------------------------------------------------|
| <b>SigLIP-2 Vision Encoder</b> | ❄️ <b>Frozen</b> ( requires_grad = False )               | ❄️ <b>Frozen</b> ( requires_grad = False , set to .eval() )       |
| <b>2-Layer MLP Projector</b>   | 🔥 <b>Trained</b> (Random init, LR = $2 \times 10^{-4}$ ) | 🔥 <b>Trained</b> (From Phase 1 weights, LR = $2 \times 10^{-5}$ ) |
| <b>LLM Base Weights</b>        | ❄️ <b>Frozen</b> ( requires_grad = False )               | ❄️ <b>Frozen</b> (Loaded in 4-bit QLoRA)                          |
| <b>LLM LoRA Adapters</b>       | N/A (No LoRA used)                                       | 🔥 <b>Trained</b> ( $r = 32, \alpha = 64$ on 7 layers)             |
| <b>Visual Token Count</b>      | 324 tokens (Pooled via 18x18 adaptive pool)              | <b>729 tokens</b> (Full 27x27 patch resolution)                   |
| <b>Sequence Length</b>         | 256 tokens                                               | 3,072 tokens                                                      |

### 8.3 LengthGroupedBatchSampler Performance Optimization

In 13\_lily-vision-phase2-sft.ipynb, multimodal sequences vary drastically in token length. Standard batching causes massive padding overhead. We implemented a custom PyTorch batch sampler that groups sequences by length:

```
class LengthGroupedBatchSampler(Sampler):
    def __init__(self, lengths, batch_size, mega_batch_mult=50, seed=42):
        self.lengths = lengths
        self.batch_size = batch_size
        self.mega_batch_size = batch_size * mega_batch_mult # 24 * 50 = 1200 samples

    def __iter__(self):
        # Partition into mega-batches, sort by sequence length, form mini-batches
        pass
```

**Optimization Outcome:** Reduced total Phase 2 SFT training duration from ~3 hours down to ~20 minutes on an A100 40GB GPU.

## 9. Cloud Checkpointing & Fault-Tolerant Auto-Resumption

Across all cloud training notebooks ( 02 , 04 , 07 , 12 , 13 ), running on serverless infrastructure (Modal) presents a risk of preemption or timeout. We built custom Hugging Face Hub callbacks that automatically snapshot checkpoints to



private repos and auto-resume on restart.

```
from huggingface_hub import HfApi, snapshot_download

# Auto-Resume Logic (07_modal_lily_1_5b_distill.ipynb)
if RESUME_FROM_CHECKPOINT and CHECKPOINT_REPO:
    api = HfApi()
    files = list(api.list_repo_files(CHECKPOINT_REPO, token=HF_TOKEN))
    ckpt_nums = [int(f.split("/")[0].split("-")[-1]) for f in files if "checkpoint-" in f]
    if ckpt_nums:
        latest = max(ckpt_nums)
        local_dir = os.path.join(OUTPUT_DIR, f"checkpoint-{latest}")
        print(f"Downloading latest checkpoint-{latest} from HF Hub...")
        snapshot_download(
            repo_id = CHECKPOINT_REPO,
            allow_patterns = [f"checkpoint-{latest}/*"],
            local_dir = OUTPUT_DIR,
            token = HF_TOKEN,
        )
        _resume_ckpt = local_dir
```

---

## Conclusion & Summary Checklist

By combining **PEFT (LoRA/QLoRA)**, **Hyperparameter Scaling Equations**, **GRPO Reinforcement Learning**, **Offline SFT Distillation**, **GGUF K-Quantization**, and **2-Phase Multimodal VLM Alignment**, our codebase establishes a complete, production-grade LLM development framework.